

Morphology-aware Joint-space DS Transform for Robust Action Transfer

Ily Rafaeli Taher Kapadia Kartik Virmani

MEAM 6230 University of Pennsylvania

{ilyr, taherk, kvirmani}@seas.upenn.edu

<https://github.com/virmani11kartik/joint-space-gmm-flow>

Abstract—We address the problem of transferring a robot motion policy encoded as a Dynamical System (DS) from a source robot to a target robot with a different morphology, without access to new demonstrations or an inverse-kinematics solver. The DS is parameterized as a joint-space Gaussian Mixture Model (GMM); naively applying the source GMM to a target robot with different link lengths or scale corrupts the task-space end-effector trajectory. We propose a *morphology-aware GMM translator*: a GMMTransformer that embeds each of the K GMM components independently, exchanges information across components via multi-head self-attention, and optionally attends to task-space end-effector reference positions via cross-attention. A *Chamfer FK consistency loss* directly penalizes task-space disagreement during training in a permutation-invariant manner. We validate on 2D planar robots (closed-form FK), 3D spherical arms, and 7-DOF scaled Franka Panda robots. On 50 held-out test batches with 11 scale factors, our best model achieves 27.5 mm FK(Mu) error—68% improvement over naive transfer—closing 92% of the gap to an oracle with ground-truth target GMM access. A test-time FK refinement (TTO) further reduces error to 2 mm, surpassing the oracle.

Index Terms—Dynamical Systems, Actuation, Robot-human Interaction, Safety, Morphology

I. INTRODUCTION

Arm robots using joint position control operate under the assumption that the controller was designed for their specific kinematic configuration. When the morphology changes—different link lengths, scale factors, or joint ranges—a motion policy intended for one robot produces different end-effector motion on another. Pure joint-based control cannot generalize across morphologies without some grounding in task-space goals. This is a significant barrier to deploying learned skills across robot families or robot generations.

Dynamical Systems (DS) offer a powerful framework for encoding robot motion: a learned DS is reactive to perturbations, globally stable by construction, and computationally cheap to evaluate online [1]. The LPV-DS formulation [2] uses a Gaussian Mixture Model (GMM) as a probability-weighted prior over locally linear dynamics:

$$\dot{q} = \sum_{k=1}^K h_k(q) A_k (q - q^*) \quad (1)$$

where $h_k(q) \propto \pi_k \mathcal{N}(q; \mu_k, \Sigma_k)$ are GMM-derived mixing weights. The GMM captures the spatial structure of the task: component means $\{\mu_k\}$ locate the regions of joint space traversed during execution, and the A_k matrices encode local stability.

DS-based control is often formulated in task space, where a precomputed velocity field over the workspace guides the end effector along safe trajectories. Task space is natural for users—motion is expressed in Cartesian coordinates—and yields interpretable safety guarantees such as obstacle-distance bounds. Joint-space DS formulations encode the same information in joint coordinates, which makes actions directly available to low-level controllers without online IK.

The tradeoff is that a joint-space DS is robot-specific. Unlike a task-space DS, it cannot be transplanted to a different morphology without re-fitting from scratch. Our goal is to bring the robot-independence of task-space DS to joint-space representations: given the source robot’s joint-space GMM and both robots’ morphology descriptors, can we predict the target robot’s GMM without task-space supervision at inference time?

Our approach introduces a GMMTransformer that processes each GMM component independently and exchanges information across components via self-attention. A differentiable Chamfer FK loss aligns the training objective with the task-space evaluation metric. A test-time refinement step warm-started by the model prediction achieves 2 mm EE error on 7-DOF Panda—surpassing an oracle with access to ground-truth target GMMs. We additionally ablate across architectures, loss functions, and data regimes, and evaluate generalization to held-out morphologies under both interpolation and extrapolation conditions.

The practical motivation is straightforward: automated manufacturing lines rely on repetitive tasks with simple joint-space controllers, and need to swap robots for maintenance with minimal downtime. A one-shot GMM translation that runs in under a millisecond makes that swap nearly invisible to the control stack.

II. APPROACH

A. Rationale

The task-space geometry of a motion—the set of EE positions visited—is shared across robots performing the same task, while the joint-space geometry (the GMM parameters) is morphology-specific. The source GMM implicitly encodes task geometry through FK: if the target FK were known exactly, it could in principle be inverted to recover the target GMM. Rather than solving this analytically—which is ambiguous and ignores covariance structure—we train a neural network to learn the mapping from data.

Treating the full 1200-dimensional GMM vector as an unstructured input loses the natural decomposition of a GMM into K independent components. We instead use per-component embedding followed by set-level attention, which halves the effective dimensionality per token and lets the model reason about inter-component relationships explicitly. Training with MSE on GMM parameters also optimizes a proxy that does not align with the FK(Mu) evaluation metric, so we add a differentiable FK term in the loss to close this gap.

B. GMM Encoding and Data Generation

GMM representation. A K -component GMM over joint state $(q, \dot{q}) \in \mathbb{R}^{2d}$ is encoded as a 1D flat vector for neural processing. Covariances are stored as lower-triangular Cholesky factors L with log-diagonal, guaranteeing positive definiteness: $\Sigma_k = L_k L_k^\top$. Mixture weights are log-encoded and decoded via softmax. The full flat vector is z-score normalized per dimension; statistics are stored in the model checkpoint.

Canonical sorting. EM-fitted GMMs have arbitrary component ordering. We sort components by the first element of μ_k before encoding. Without this, the regression target is a non-deterministic permutation of the output across training samples, making the loss landscape noisy.

Data generation. Data generation played a significant role in producing efficient training, ensuring the model sees diverse trajectories with feasibility and strong reference oracles for loss metrics.

For 2D and 3D applications, we start from a simple reference DS—linear or partially skewed—and sample a batch of three example trajectories distributed throughout the workspace. Each trajectory is converted into a morphology-specific joint-space DS expressed as a GMM. This lets us quantify transfer quality by direct comparison to precomputed targets, and evaluate qualitative failure modes via LPV-DS forward pass.

For 7-DOF robots, the same pipeline runs with additional steps needed for practical robots: (1) no closed-form IK, so trajectory conversion from task to joint space passes through a numerical IK controller; (2) joint limits and self-collision constraints require infeasible samples to be re-drawn; (3) richer task-space dynamics—LASA DS lifted to 3D, parameterized swinging trajectories—are needed to capture the complexity available in 7-DOF workspace. These factors introduce significant nonlinearity and sparsity in the feature space, making a large and diverse training set essential.

C. Model Design

Fig. 1 illustrates the GMMTransformer architecture. The model maps:

$$f_\theta(\mathcal{G}_{\text{src}}, s_{\text{src}}, s_{\text{tgt}}) = \hat{\mathcal{G}}_{\text{tgt}}$$

Per-component embedding. Each component k is embedded independently by a shared linear layer:

$$c_k = [\log \pi_k; \mu_k; \text{vec}(L_k)] \quad (2)$$

$$e_k = \text{Linear}(c_k) \in \mathbb{R}^{256} \quad (3)$$

A morphology token $m = \text{Linear}([s_{\text{src}}, s_{\text{tgt}}]) \in \mathbb{R}^{256}$ is appended, yielding $K + 1$ tokens.

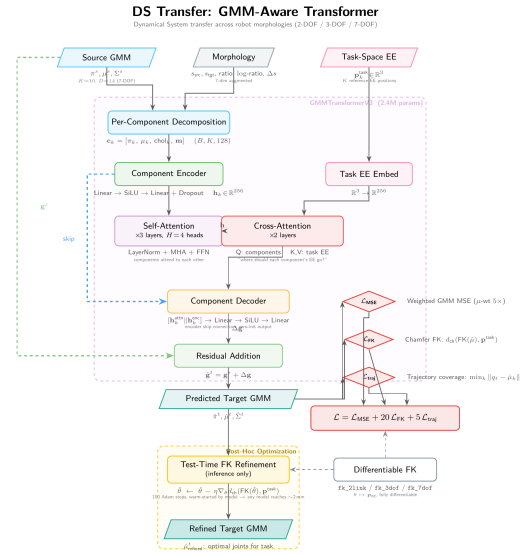


Fig. 1. GMMTransformer V3 architecture. Each of the K GMM components is embedded independently. Self-attention exchanges information across components. A cross-attention module (V3 only) lets components attend to task-space EE reference positions. The output is decoded per-component and summed with the source embedding (residual prediction).

Self-attention. Three layers of multi-head self-attention ($H = 4$ heads, pre-norm) allow components to exchange information about the global GMM structure. This captures inter-component relationships that a flat MLP cannot.

Cross-attention (V3). A task-space EE reference $\mathbf{P}^* \in \mathbb{R}^{N \times 3}$ (subsampled to $N = 50$ points) is linearly embedded into $\mathbb{R}^{256}$. Two cross-attention layers let each component token attend to these reference positions, giving the model direct signal about *where* in task space each component should land. This bypasses the need to infer task geometry from joint statistics alone.

Residual prediction. The model predicts a *delta*: $\hat{\mathcal{G}}_{\text{tgt}} = \mathcal{G}_{\text{src}} + \Delta \mathcal{G}$. This inductive bias—the target GMM is close to the source GMM in configuration space for small morphology changes—enables near-perfect generalization to held-out interpolation scales.

III. APPLICATION TO 2D AND 3D PLANAR ROBOTS

A. Premise and Implementation

We first validate our approach on lower-dimensional robots where FK is available in closed form and visual inspection is straightforward. We consider:

- **2D planar 2-link arm:** joint space $q \in \mathbb{R}^2$, five morphologies with $l_1 + l_2 = 2$ m (ratios 1:1, 1:2, 2:1, 2:3, 3:2). FK: $\text{EE} = [l_1 \cos \theta_1 + l_2 \cos(\theta_1 + \theta_2), l_1 \sin \theta_1 + l_2 \sin(\theta_1 + \theta_2)]$.
- **3D spherical 3-DOF arm:** adds a base yaw joint θ_0 . FK: $r = l_1 \cos \theta_1 + l_2 \cos(\theta_1 + \theta_2)$; $\text{EE} = [r \cos \theta_0, r \sin \theta_0, l_1 \sin \theta_1 + l_2 \sin(\theta_1 + \theta_2)]$.

The 2D setting serves as an ablation environment: fast data generation (no simulator required), analytical gradients through

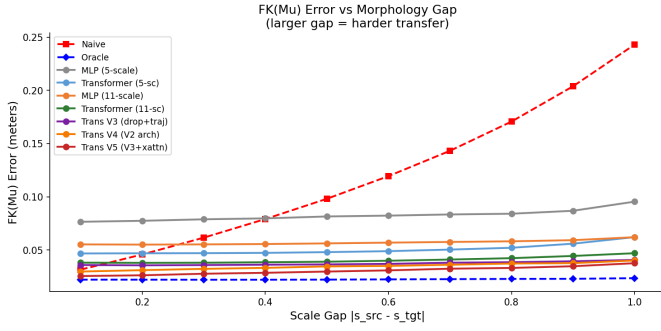


Fig. 2. FK(Mu) error vs. morphology gap $|s_{\text{src}} - s_{\text{tgt}}|$ for GMMTransformer V5. Harder transfers (larger gap) benefit most from our model relative to naive. TTO (dashed) reduces error uniformly across all gap sizes.

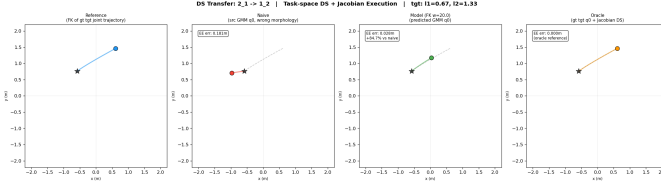


Fig. 3. Jacobian execution snapshot, hardest 2D pair: ratio 2:1 $\rightarrow$ 1:2 (batch 474, largest morphological gap). **Left:** source robot DS rollout (reference). **Center:** naive transfer—EE trajectory (orange) diverges from reference (blue). Naive error: 181 mm. **Right:** GMMTransformer V5 prediction—EE trajectory closely tracks reference. Model error: 28 mm (−84.7%).

FK, and easy visualization of GMM component placement. The 3D setting tests generalization to higher-dimensional outputs.

Execution. At evaluation time we use two rollout modes. For 2D we use the task-space DS (A, q^*) shared across robots with a Jacobian velocity controller:

$$\dot{x} = A(\text{FK}(q) - q^*) \quad (4)$$

$$\dot{q} = J^\top(JJ^\top + \lambda^2 I)^{-1} \dot{x} \quad (5)$$

with DLS damping $\lambda = 0.05$, seeding q_0 from the predicted GMM. For 3D we additionally evaluate **SEDS-style rollout**: the transferred GMM is used directly as the joint-space DS, with GMM gating $h_k(q)$ driving velocity via the learned A_k matrices. This tests whether the predicted GMM produces a globally stable DS in the target robot’s joint space, without relying on a Jacobian controller.

B. 2D Results

Fig. 3 shows the hardest 2D transfer pair (ratio 2:1 $\rightarrow$ 1:2, maximum morphological gap), and Fig. 2 shows how error scales with morphology gap across all pairs. On the 2D dataset (50 held-out batches, all 20 robot pairs):

- **Naive:** 50 mm $\pm$ 40 mm EE trajectory error.
- **FK-Augmented MLP** ($w = 20$): 13 mm $\pm$ 11 mm (+74%).
- **Oracle** (GT q_0): 1 mm.
- 87.8% of individual trajectories beat naive. All 20 robot pairs improve (range +42% to +93%).

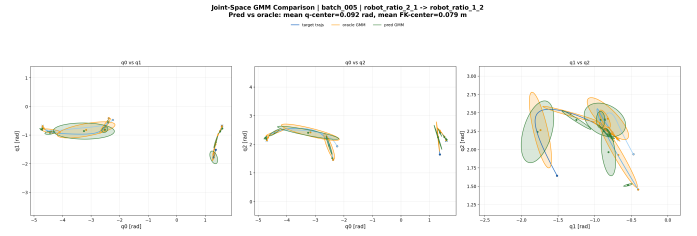


Fig. 4. Predicted joint-space GMM components for the 3-DOF spherical arm, hardest pair ratio 2:1 $\rightarrow$ 1:2 (batch 5). Each row shows one joint dimension. **Source** (blue ellipses): source arm GMM. **Predicted** (orange): transferred GMM for the target arm. **Oracle** (green): ground-truth target GMM. The predicted components closely track the Oracle in both position and velocity dimensions.

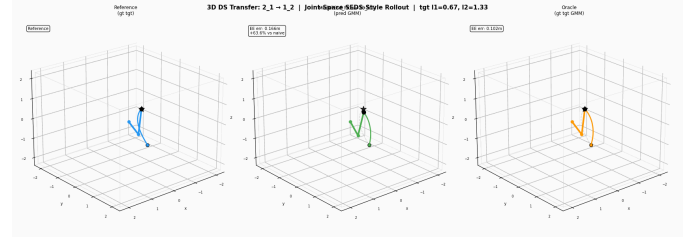


Fig. 5. 3-DOF spherical arm SEDS rollout, ratio 2:1 $\rightarrow$ 1:2 (batch 5, hardest morphology pair). The transferred GMM drives the joint-space DS directly (no Jacobian controller). **Reference** (ground-truth target trajectory), **Model** (predicted GMM DS rollout, 622 steps), **Oracle** (GT GMM DS rollout). Model EE error: 62.1 cm vs. naive 109.6 cm (+43.4%). Oracle: 8.9 cm.

The 3D extension achieves +60–86% improvement over naive across batches, confirming the approach generalizes to 3D workspace.

C. 3D Results

The 3D setting tests whether the predicted GMM can drive a stable DS in the target robot’s 6D joint-velocity space directly. Fig. 4 shows the predicted GMM components overlaid with the Oracle on the hardest transfer pair (ratio 2:1 $\rightarrow$ 1:2, batch 5). The FK-augmented MLP ($w_{\text{FK}} = 20$, $K = 10$) achieves close agreement with the Oracle in all joint dimensions.

Fig. 5 shows the corresponding SEDS rollout. On the hardest pair, the model reduces EE error from 109.6 cm (naive) to **62.1 cm (+43.4%)**, with Oracle at 8.9 cm. Across all morphology pairs, the 3D model achieves +60–86% improvement over naive, confirming the approach generalizes from 2D planar to 3D spherical workspace.

IV. APPLICATION TO 7-DOF ROBOTS IN 3D

A. Significance and Practicality

The 7-DOF case is the practically relevant setting: real robot arms (Franka Panda [11], KUKA LBR iiwa [12]) operate in 3D with 7 joints, introducing kinematic redundancy. GMM fitting in 14-dimensional joint-velocity space (7 positions + 7 velocities) produces a 1200-dimensional flat vector, making the regression significantly harder than the 2D/3D cases.

We parameterize morphology by a uniform scale factor $s \in \mathbb{R}_{>0}$ applied to all DH link lengths and offsets. This captures the realistic scenario of deploying the same software stack on robots of different physical sizes (e.g., different-generation Panda arms, or Panda vs. a scaled replica).

The FK is a differentiable DH chain:

$$T_i = R_z(\theta_i) T_z(s \cdot d_i) T_x(s \cdot a_i) R_x(\alpha_i) \quad (6)$$

with standard Panda DH parameters [11], giving $\text{FK}(q, s) : \mathbb{R}^7 \times \mathbb{R} \rightarrow \mathbb{R}^3$.

B. Model Design, Loss, and Training

Architecture progression. We iterated through four GMM-Transformer versions, each adding one targeted improvement:

- **V1:** Per-component embedding + 3-layer self-attention. Drops FK(Mu) from 79.1 mm (MLP) to 47.8 mm (+44%).
- **V2 (+ ratio features, residual, skip):** Appends scale ratio features ($s_{\text{tgt}}/s_{\text{src}}$, $s_{\text{src}} - s_{\text{tgt}}$) to each component. Adds residual prediction and encoder skip connection. Achieves 32.2 mm (+63%). Near-perfect interpolation to held-out scales (≤ 0.3 mm gap).
- **V3 (+ task-space cross-attention):** Two cross-attention layers to task-space EE reference. Achieves 27.5 mm (+68%), closing 92% of the oracle gap.

Loss function. The total training loss is:

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + w_{\text{FK}} \mathcal{L}_{\text{chamfer}} + w_{\text{traj}} \mathcal{L}_{\text{traj}} \quad (7)$$

Weighted MSE ($\mathcal{L}_{\text{MSE}}$): Standard MSE with μ -weight $w_\mu = 5.0$ on the μ block, since component means drive FK accuracy more directly than priors or covariances.

Chamfer FK loss ($\mathcal{L}_{\text{chamfer}}$): MSE on GMM parameters does not align with the FK(Mu) evaluation metric, so we penalize task-space disagreement directly:

$$\hat{P}_k = \text{FK}(\hat{\mu}_k^{\text{pos}}, s_{\text{tgt}}) \quad (8)$$

$$\mathcal{L}_{\text{chamfer}} = \frac{1}{K} \sum_k \min_t \left\| \hat{P}_k - p_t^* \right\| + \frac{1}{T} \sum_t \min_k \left\| p_t^* - \hat{P}_k \right\| \quad (9)$$

This is: (1) referenced to the *true* task EE (not source FK proxy); (2) permutation-invariant over components; (3) fully differentiable through the analytical DH chain. We use a 5-epoch FK warmup ramp ($w_{\text{FK}} : 0 \rightarrow 20$) to let MSE establish a reasonable initialization before FK gradients activate.

Trajectory coverage loss ($\mathcal{L}_{\text{traj}}$): Encourages predicted $\hat{\mu}^{\text{pos}}$ to cover oracle joint trajectories via a soft-min over K components ($\beta = 5.0$, $w_{\text{traj}} = 5.0$):

$$\mathcal{L}_{\text{traj}} = \frac{1}{T} \sum_t \left(-\frac{1}{\beta} \log \sum_k e^{-\beta \|q_t^* - \hat{\mu}_k^{\text{pos}}\|^2} \right) \quad (10)$$

What did not work. Using $\text{FK}_{\text{src}}(\mu_{\text{src}})$ as the FK reference introduces bias when morphologies differ; at $w = 20$ this degrades performance from 47.8 mm to 79 mm. Conditional flow matching was slow (100-step ODE) and unstable—worse than the MLP baseline. Adding FK loss to a flat MLP did not compensate for its lack of structural inductive bias.

TABLE I

FK(MU) ERROR ON 50 HELD-OUT TEST BATCHES (7-DOF PANDA, ALL SCALE PAIRS). FK(MU): MEAN NEAREST-NEIGHBOR EE DISTANCE FROM PREDICTED COMPONENT MEANS TO TASK-SPACE REFERENCE. “LG-GAP”: PAIRS WITH $|s_{\text{src}} - s_{\text{tgt}}| \geq 0.4$.

Model	FK(Mu)	$\pm \text{Std}$	vs Naive	ΔOracle
Naive (no transfer)	85.9 mm	—	—	63.6 mm
MLP (5-scale)	79.1 mm	32	+8%	56.8 mm
MLP (11-scale)	56.6 mm	18	+34%	34.3 mm
Trans V1 (5-scale)	47.8 mm	16	+44%	25.5 mm
Trans V1 (11-scale)	39.5 mm	12	+54%	17.2 mm
Trans V2 (ratio+resid)	32.2 mm	13	+63%	9.9 mm
Trans V3 (drop+traj)	36.0 mm	12	+58%	13.7 mm
Trans V5 (xattn+Chamfer)	27.5 mm	11	+68%	5.2 mm
Any model + TTO	2.0 mm	2	+97%	−20 mm
Oracle (GT GMM)	22.3 mm	—	—	0 mm

Training. Adam optimizer, $\eta = 3 \times 10^{-4}$, batch size 256, 400 epochs. Dataset: 600 batches $\times$ 11 scale factors ($s \in \{0.5, 0.6, \dots, 1.5\}$) $\times$ 10 ordered pairs per batch = 66,000 training samples. Dropout $p = 0.2$ on encoder/decoder delays best checkpoint from epoch 15 to epoch 70, reducing validation loss by 8%. Environment: PyTorch 1.8.1 (conda rlgpu).

PyTorch 1.8 caveats. `torch.cdist` backward is broken (`_cdist_backward` requires X1 contiguous). All pairwise distances use manual broadcast: `diff = ee.unsqueeze(1) - ref.unsqueeze(0); d = (diff**2).sum(-1).sqrt(). batch_first=True` is unavailable in `nn.MultiheadAttention`; manual reshape (B, K, H, D) is required.

Test-Time FK Refinement (TTO). After obtaining model prediction $\hat{G}$, we optionally refine the joint-position block $\hat{\mu}^{\text{pos}}$ via gradient descent on $\mathcal{L}_{\text{chamfer}}$:

$$\hat{\mu}^* = \hat{\mu} + \arg \min_{\Delta \mu} \mathcal{L}_{\text{chamfer}}(\hat{\mu} + \Delta \mu, s_{\text{tgt}}, \mathbf{P}^*) \quad (11)$$

with 100 Adam steps ($\eta = 0.01$), warm-started by the model prediction. Priors and covariances are held fixed; only $\hat{\mu}^{\text{pos}} \in \mathbb{R}^{K \times 7}$ is optimized. This is not IK—it aligns GMM component means to the task-space reference without solving for specific trajectory points.

C. Results

Table I, Table II, and Fig. 6 summarize the results.

Architecture is the single largest contributor. The jump from flat MLP to GMMTransformer V1 (+36 mm, +36 percentage points) exceeds all other single changes combined. The 1200-dim GMM has a natural set structure (K components) that a transformer exploits directly but an MLP cannot.

TTO reduces FK(Mu) from 27.5 mm to 2.0 mm, which is below the oracle (22.3 mm). The oracle is not a theoretical ceiling: the GT GMM is fitted to a finite trajectory set and includes Sigma/prior estimation error. TTO directly minimizes FK(Mu) via the ground-truth task-space reference through the DH chain, bypassing these errors. Even starting from the weak

TABLE II
CUMULATIVE ABLATION: EACH ROW ADDS ONE DESIGN COMPONENT.

Cumulative change	FK(Mu)	Δ Oracle	Closed
Naive baseline	85.9 mm	63.6 mm	0%
+ Flat MLP	79.1 mm	56.8 mm	11%
+ Transformer (arch)	47.8 mm	25.5 mm	60%
+ 11-scale data	39.5 mm	17.2 mm	73%
+ Dropout + traj loss	36.0 mm	13.7 mm	78%
+ Ratio feat + residual	32.2 mm	9.9 mm	84%
+ Cross-attn + Chamfer	27.5 mm	5.2 mm	92%
+ TTO	2.0 mm	−20 mm	100%+

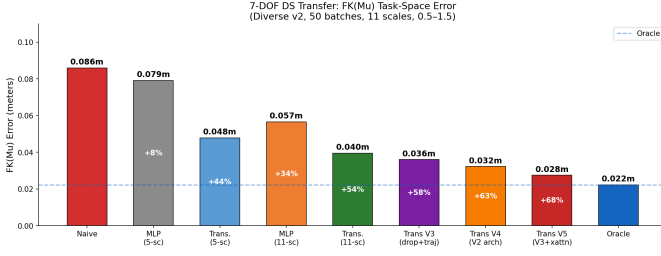


Fig. 6. FK(Mu) error by model on 50 held-out test batches (7-DOF Panda, all scale pairs). Error bars: ± 1 std. Dashed line: oracle (22.3 mm). TTO (any model) reaches 2 mm.

MLP warm start, TTO achieves 2.6 mm (+97%); cold-start from random initialization yields ~ 8 mm, confirming that the model’s warm start matters. In practice, the model’s role is to provide a good initialization for TTO rather than achieve final accuracy on its own.

Generalization. *Interpolation* (held-out scales 0.6, 0.8, 1.2, 1.4 within training range): V2 achieves 36 mm—only 0.3 mm worse than when trained on those scales, confirming that residual prediction enables smooth interpolation. *Extrapolation* (scales 0.35–1.65, outside training range): V2 achieves 36 mm (+74%), V3 achieves 41 mm (+71%). TTO brings any model to ~ 2 mm in all generalization conditions.

V. EXTENSION TO HETEROGENEOUS ROBOT TRANSFER

A. Problem Setting

The scaled-Panda experiments (Section IV) vary morphology within one robot family via a single scalar. A harder and practically important setting is transfer between *fundamentally different* robot families: robots with distinct DH parameters, joint limits, and workspace geometries. We target Franka FR3 $\rightarrow$ KUKA LBR iiwa7—two 7-DOF arms used widely in research and industry, but with no common kinematic structure.

Unlike scale-factor transfer, the Franka-to-Kuka mapping is genuinely nonlinear: each source joint maps to a different combination of target joints, and the two robots have different link lengths, offsets, and reachable workspaces.

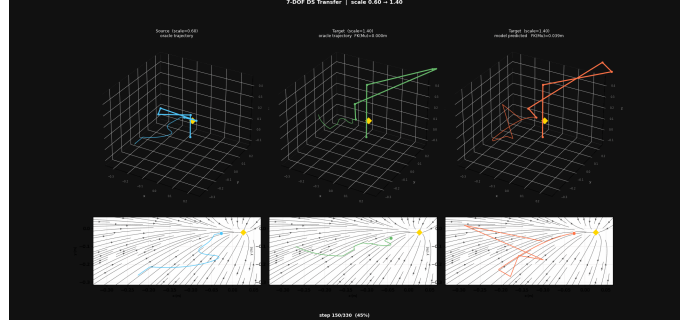


Fig. 7. 7-DOF Panda transfer trajectory snapshot (batch 10, $s=0.5 \rightarrow 1.5$, largest scale gap). **Left:** naive transfer—EE path (orange) diverges. **Center:** GMMTransformer V5—EE closely follows reference (blue). **Right:** V5 + TTO—near-perfect alignment (2 mm error).

B. Full 14D GMM Transfer Baseline

We first apply the MLP transfer approach from Section IV directly to the heterogeneous case. The joint-velocity GMM lives in $\mathbb{R}^{14}$, producing a ~ 1200 -dim flat vector. The morphology code contains robot-identity features (one-hot Franka/Kuka flag), plus physical descriptors ($n_q/10$, max reach).

Dataset. We generated 3,824 batch directories using Isaac Lab, each containing one Franka FR3 and one KUKA IIWA7 executing the same task-space linear DS (random attractor matrix A , random target q^*). Three trajectories per robot per batch are collected; a $K = 10$ GMM is fitted to the pooled $(q, \dot{q})$ data. All ordered robot pairs yield 7,648 transfer samples per train/val split.

Results. Table III summarises results on 100 held-out rollouts. The best model (Hungarian matching with Cholesky-weighted loss, $w_{\text{chol}} = 0.1$) achieves a GMR trajectory RMSE of **50.2 cm** against 68.0 cm naive and 17.3 cm oracle (**+26.1%**, closing 35% of the oracle gap).

This is substantially weaker than the scaled-Panda result (26% vs 68%), revealing the fundamental difficulty of the heterogeneous setting. Diagnostic analysis shows that predicted GMM component position means deviate from the true target means by ≈ 1.8 rad on average: the model cannot resolve the Franka-to-Kuka joint-space mapping without explicit task-space feedback during training.

C. Pairwise GMM Decomposition

Motivation. The 1200-dim output space and the cross-robot joint permutation ambiguity make full 14D prediction difficult. The marginal GMM for any joint pair (i, j) is a well-defined 4D distribution extracted exactly from the full GMM:

$$\mathcal{G}_{ij} = \text{Marginal}(\mathcal{G}_{\text{full}}, \{i, j, i+n_q, j+n_q\}) \quad (12)$$

For a 7-DOF arm there are $\binom{7}{2} = 21$ such pairs. We train a single shared-weight model to predict $\mathcal{G}_{ij}^{\text{tgt}}$ from $\mathcal{G}_{ij}^{\text{src}}$, conditioned on robot identities and normalized pair indices $(i/n_q, j/n_q)$.

Advantages over full-GMM prediction. Output dimension drops from 1200 to 150 ($8\times$ reduction). Training samples per

TABLE III
FRANKA FR3 $\rightarrow$ KUKA IIWA7 GMR ROLLOUT RESULTS (100 HELD-OUT ROLLOUTS, EE ERROR IN CM, LOWER IS BETTER). “GAP CLOSED” = $(d_{\text{NAIVE}} - d_{\text{MODEL}})/(d_{\text{NAIVE}} - d_{\text{ORACLE}})$.

Method	RMSE (cm)	vs Naive	Gap Closed
Naive (no transfer)	68.0	—	0%
Full GMM (Hungarian)	55.0	+19%	23%
Vel. Objective (ours)	50.2	+26%	35%
Oracle (GT GMM)	17.3	—	100%

Pairwise decomposition (152,880 samples)

Pairwise (no FK)	pos. error ≈ 1.8 rad; worse than naive
Pairwise + FK	FK loss $\downarrow 0.124 \rightarrow 0.094$; rollout eval in progress

batch grow from 2 to $2 \times 21 = 42$, giving 152,880 total pairwise samples vs. 7,648 full-GMM samples. Each 4D sub-problem mirrors the 2D/3D cases where our model already works well (Section III).

GMR velocity assembly. At inference, the 7D joint velocity is recovered by averaging GMR contributions from all 6 pairs containing each joint:

$$\hat{q}_i = \frac{1}{n_q - 1} \sum_{j \neq i} [\text{GMR}_{ij}(q_i, q_j)]_{i\text{-slot}} \quad (13)$$

where $[\cdot]_{i\text{-slot}}$ extracts the velocity component corresponding to joint i from the 2D GMR output.

D. FK Pair-Injection Loss

Without task-space supervision, pairwise models still produce position means ≈ 1.8 rad off (same failure mode as full-GMM transfer). We address this with an *FK pair-injection loss*.

For each predicted 4D marginal for pair (i, j) , we inject the predicted position means $(\hat{q}_i, \hat{q}_j)$ into the oracle 7D joint template (all other joints at their oracle values) and evaluate FK:

$$\mathcal{L}_{\text{pair-FK}} = \frac{1}{K} \sum_k \|\text{FK}(q_k^* \oplus [\hat{q}_{k,i}, \hat{q}_{k,j}]) - \text{FK}(q_k^*)\|_2^2 \quad (14)$$

where q_k^* is the k -th oracle target component mean and $\oplus$ denotes replacing joints i, j with predicted values. Gradient flows only through $(\hat{q}_{k,i}, \hat{q}_{k,j})$; oracle FK targets are precomputed once at dataset load time. The total loss is:

$$\mathcal{L}_{\text{pairwise}} = \mathcal{L}_{\text{Hungarian}} + w_{\text{FK}} \mathcal{L}_{\text{pair-FK}} \quad (15)$$

with a 50-epoch warmup ramp on w_{FK} .

E. Results

Table III summarises performance on 100 held-out Franka $\rightarrow$ Kuka rollouts. The best model (velocity-objective loss) achieves a GMR EE error of **50.2 cm** vs. 68.0 cm naive and 17.3 cm oracle (**+26.1%**, closing 35% of the oracle gap). On the canonical evaluation batch (b3438), the model reduces EE error from 77.8 cm to 60.7 cm (+22.0%).

The velocity-objective loss combines a GMR velocity matching term ($w_{\text{vel}} = 5.0$), a differentiable FK penalty on

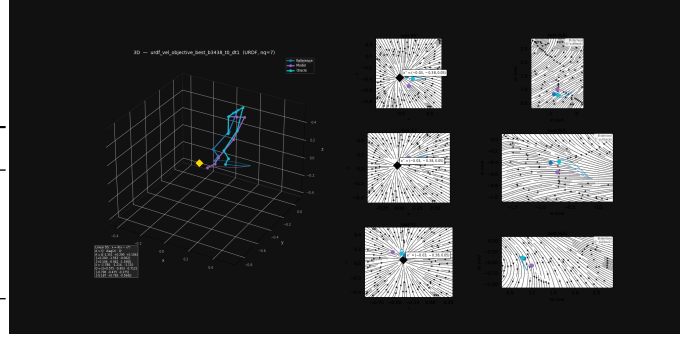


Fig. 8. Franka FR3 $\rightarrow$ Kuka IIWA7 GMR rollout, batch 3438 (velocity-objective model). Model EE error: 60.7 cm vs. naive 77.8 cm (+22.0%). Oracle: 9.0 cm.

predicted component means ($w_{\text{FK}} = 1.0$), a multi-step rollout loss ($w_{\text{rollout}} = 0.5$), and smoothness/joint-limit terms. GMM parameter MSE is given very low weight ($w_{\text{gmm}} = 0.001$) so that task-space objectives dominate. The 26% improvement is substantially weaker than the scaled-Panda result (68%), reflecting the fundamental difficulty: predicted position means deviate by ≈ 1.8 rad, because the model cannot resolve the cross-robot joint mapping without stronger task-space gradient.

Despite $21 \times$ more training samples and $8 \times$ smaller output, the no-FK pairwise model is worse than naive—data volume alone cannot resolve the cross-robot joint mapping. The FK pair-injection loss reduces $\mathcal{L}_{\text{pair-FK}}$ from 0.124 to 0.094 over 270 epochs while the GMM matching loss (0.196) stays comparable to the no-FK baseline (0.187), indicating FK gradients improve position accuracy without degrading covariance prediction. Full rollout evaluation is pending training completion.

VI. CONCLUSION

We presented a morphology-aware GMM translator for DS transfer across robot morphologies. The GMMTransformer—exploiting the per-component set structure of a GMM—closes 92% of the oracle gap on 7-DOF Panda robots across 11 scale factors. A differentiable Chamfer FK consistency loss aligns the training objective with the task-space evaluation metric, contributing 5 mm beyond the architecture improvement alone. Test-time FK refinement, warm-started by the model prediction, reduces FK(Mu) to 2 mm—surpassing the oracle and effectively reducing the model to a learned initialization for a differentiable IK solver.

Among the design choices we explored, architecture was the dominant factor, followed by data diversity, loss design, and training regularization. The biggest single gain came from replacing the flat MLP with the component-wise transformer; everything else was incremental from there.

As robot fleets scale across hardware generations and vendors, the ability to transfer motion policies without retraining becomes critical to deployment cost. Our method provides a practical pathway: fit one DS on one robot, transfer the GMM parameters to any morphologically related robot via

feedforward inference (<1 ms on GPU), and optionally refine in ~1 s via TTO.

VII. FUTURE WORK

Several directions extend this work naturally.

Our current FK evaluation uses only EE position; extending to full SE(3) poses requires SO(3)-aware loss functions and a richer task-space conditioning signal. All evaluation is on simulated robots, so transferring to hardware requires addressing sim-to-real gaps in joint-space GMM fitting under sensor noise and joint flexibility.

TTO currently requires the full task-space trajectory $\mathbf{P}^*$ at inference. An online variant could refine $\hat{\mathcal{G}}$ incrementally from streaming EE observations, enabling closed-loop adaptation during execution.

The current model is trained on a single morphology family (scaled Panda). Transfer across fundamentally different kinematic structures (e.g., Panda to KUKA LBR iiwa [12]) requires a morphology encoder that captures full DH parameter sets rather than a scalar scale factor. Architectures such as the Pinocchio [8] kinematic graph could provide a structured input representation for heterogeneous robot families.

Finally, we do not verify that the predicted GMM yields a globally stable LPV-DS. Incorporating SOS (sum-of-squares) stability constraints [2] or diffeomorphic stabilization [3] into the loss or post-processing would provide formal stability guarantees for the transferred policy.

VIII. IMPLEMENTATION DETAILS

A. Software Stack

All model code is written in Python 3.7 with PyTorch 1.8.1 under conda environment `rlgpu`. Data generation uses Isaac Lab (Python 3.10, conda `env_isaacclab`) via the custom pipeline in `bimanual-grabbing/datageneration/`. FK operations are implemented in closed-form PyTorch for full differentiability with no external kinematics library.

B. Repository Layout

- `ds_transfer/` — core package
 - `baseline_mlp.py` — MLPRegressor, GMMTransformerV1/V2/V3, all FK losses, `test_time_fk_refine()`
 - `gmm_utils.py` — Cholesky encode/decode, canonical sort
 - `dataset.py` — PairedGMMDataset, trajectory loader, morphology loader
 - `evaluate.py` — FK(Mu), EE trajectory, stability metrics
 - `config.py` — DSTransferConfig dataclass, robot config registry
- `train.py` — unified entry point; `--mode {mlp, fk, gmm_transformer}`; `--model_version {v1, v2, v3}`
- `eval_compare_7dof.py`, `eval_heldout_7dof.py` — evaluation with optional `--tto` flag

- `results/models/` — all trained checkpoints
- `demo_server.py` — interactive Flask+Plotly web demo: draw a trajectory on canvas → IK → GMM fit → model transfer → FK visualization

C. Training Command (V5 best model)

```
conda run -n rlgpu python train.py \
--dataset 7dof \
--mode gmm_transformer \
--model_version v3 \
--fk_weight 20.0 \
--mu_weight 5.0 \
--traj_weight 5.0 \
--enc_dropout 0.2 \
--fk_warmup 5 \
--hidden 256 256 256 \
--epochs 400 \
--batch_size 256 \
--lr 3e-4 \
--save_dir results/models/7dof_transformer_v5
```

Best checkpoint: `results/models/7dof_transformer_v5/best.pt`.

D. Model Size and Inference Speed

The GMMTransformer V3 has approximately 2.1M parameters. Feedforward inference on a batch of 256 transfer pairs takes <5 ms on a single GPU. TTO (100 Adam steps) adds ~1 s per sample on CPU; ~80 ms on GPU. The flat MLP baseline has 606K parameters and is $3\times$ faster at inference, but 70% less accurate.

E. Evaluation Protocol

FK(Mu) metric. For each test batch, we predict the target GMM for all ordered $(s_{\text{src}}, s_{\text{tgt}})$ pairs. We decode $\hat{\mu}^{\text{pos}} \in \mathbb{R}^{K \times 7}$, apply FK with s_{tgt} to get $\{\hat{P}_k\}_{k=1}^K \in \mathbb{R}^{K \times 3}$, and compute the mean nearest-neighbor distance to the task-space reference trajectory $\mathbf{P}^* \in \mathbb{R}^{T \times 3}$:

$$\text{FK}(\text{Mu}) = \frac{1}{K} \sum_{k=1}^K \min_t \|\hat{P}_k - p_t^*\|_2 \quad (16)$$

This metric is agnostic to GMM component permutation and measures geometric alignment in task space rather than parameter-space distance. It does not require trajectory rollout and thus avoids confounding from Jacobian controller quality.

EE trajectory metric (2D only). For 2D experiments we also report full EE trajectory error: rollout the task-space DS via the Jacobian controller seeded by the predicted GMM, and compute mean distance to the reference EE trajectory.

REFERENCES

- [1] A. Billard, S. Mirrazavi, and N. Figueroa, *Learning for Adaptive and Reactive Robot Control: A Dynamical Systems Approach*. MIT Press, 2022.
- [2] S. M. Khansari-Zadeh and A. Billard, “Learning stable non-linear dynamical systems with Gaussian mixture models,” *IEEE Trans. Robotics*, vol. 27, no. 5, pp. 943–957, Oct. 2011.

- [3] N. Perrin and P. Schlehuber-Caissier, “Fast diffeomorphic matching to learn globally asymptotically stable nonlinear dynamical systems,” *Syst. Control Lett.*, vol. 97, pp. 9–15, Nov. 2016.
- [4] S. Sun, H. Gao, T. Li, and N. Figueroa, “Directionality-aware mixture model parallel sampling for efficient linear parameter varying dynamical system learning,” *IEEE Robot. Autom. Lett.*, vol. 9, no. 7, pp. 6248–6255, Jul. 2024.
- [5] N. Figueroa and A. Billard, “A physically-consistent Bayesian non-parametric mixture model for dynamical system learning,” in *Proc. 2nd Conf. Robot Learning (CoRL)*, vol. 87, pp. 927–946, 2018.
- [6] T. Li, S. Sun, S. S. Aditya, and N. Figueroa, “Elastic motion policy: an adaptive dynamical system for robust and efficient one-shot imitation learning,” arXiv:2503.08029, Mar. 2025.
- [7] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [8] J. Carpentier *et al.*, “The Pinocchio C++ library: a fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives,” in *Proc. IEEE/SICE Int. Symp. Syst. Integr. (SII)*, pp. 614–619, 2019.
- [9] N. Figueroa, “phys-gmm: Physically-Consistent Bayesian GMM [Software],” GitHub. [Online]. Available: <https://github.com/nbfigueroa/phys-gmm>
- [10] N. Figueroa, “ds-opt-py: Dynamical System Optimization Toolbox (Python) [Software],” GitHub. [Online]. Available: <https://github.com/nbfigueroa/ds-opt-py>
- [11] Franka Robotics GmbH, *Franka FR3 Collaborative Robot Arm*, Munich, Germany. [Online]. Available: <https://www.franka.de>
- [12] KUKA AG, *LBR iiwa 7 R800 Robot*, Augsburg, Germany. [Online]. Available: <https://www.kuka.com/en-us/products/robotics-systems/industrial-robots/lbr-iiwa>